Load Test Report

Game catalog search API · Scenario: smoke · Run: 2026-10-23--22-44-20 · terraform-first-run · Executed: 2026-10-23 22:44:41 UTC · Generated: 2026-10-23 22:46 UTC

Scenario

1 user for 1 minute: checks that the API works before adding load (~30 requests).

Setting	Value
Load profile	1 virtual user(s), constant, for 1m
Duration	66 s (2026-10-23 22:44:41 UTC -> 2026-10-23 22:45:47 UTC)
Think time	1 s between requests per user
Request timeout	30s
Endpoint	POST /search
Test queries	queries.json
Thresholds	http_req_failed: rate<0.01; http_req_duration: p(95)<5000
Authentication	IAM (AWS Signature V4)
Scenario source	copy saved with the run

Test environment and data sources

Item	Value
k6 version	v0.54.0
Container image	[redacted].dkr.ecr.us-east-1.amazonaws.com/game-catalog-k6:latest
Image ID	[redacted]
Task definition	game-catalog-k6-task:3
Compute	FARGATE · 1 vCPU · 2048 MB · us-east-1e
Run folder	s3://game-catalog-loadtest/runs/smoke/2026-10-23--22-44-20
Files	k6-summary.json: yes, run-info.json: yes, k6-results.csv: yes, scripts/ (scenario copy): yes

Data quality notes

- Small sample (23 requests): p95 and p99 depend on very few requests and can change a lot between runs.

Results

23	0.35/s	0.00 %	100.0 %
Requests	Throughput	Error rate	Checks passed

1,314 ms	3,789 ms	6,868 ms	7,735 ms
Median (p50)	p95	p99	Max

Threshold	Value	Rule	Status
Failed requests	0.00 %	rate<0.01	**OK**
Request duration (latency)	3,789 ms	p(95)<5000	**OK**

Metrics

Latency metrics, in milliseconds. Each request is split into phases: blocked, connecting, TLS handshake, sending, waiting (server processing) and receiving.

Metric	What it measures	Avg	p50	p95	Max
Request duration (latency)	Total time of a request: sending + waiting + receiving.	1,850 ms	1,314 ms	3,789 ms	7,735 ms
Waiting time (TTFB)	Server processing time until the first byte of the response.	1,850 ms	1,314 ms	3,789 ms	7,734 ms
Blocked time	Time waiting for a free connection before sending.	0.41 ms	0.00 ms	0.00 ms	9.33 ms
Connecting time	Time to open the TCP connection.	0.04 ms	0.00 ms	0.00 ms	0.90 ms
TLS handshake time	Time to negotiate the HTTPS connection.	0.10 ms	0.00 ms	0.00 ms	2.34 ms
Sending time	Time to send the request body.	0.15 ms	0.10 ms	0.35 ms	0.61 ms
Receiving time	Time to download the response.	0.13 ms	0.09 ms	0.33 ms	0.68 ms
Iteration duration	One full loop of a virtual user, including the pause.	2,852 ms	2,316 ms	4,790 ms	8,736 ms

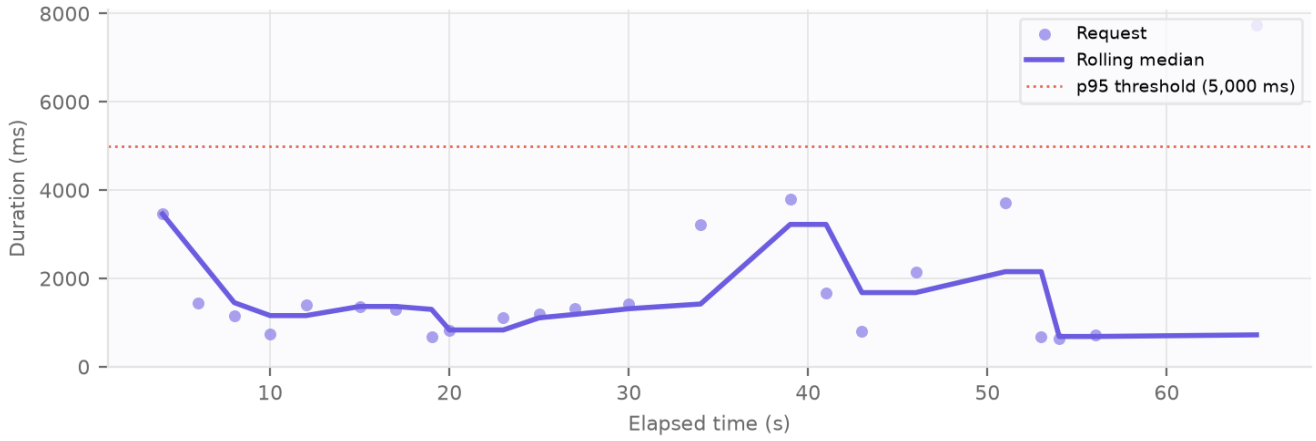
Metric	What it measures	Value
HTTP requests	Requests sent to the API during the test.	23 (0.35 per second)
Failed requests	Share of requests with an error status or no response.	0.00 %
Checks	Share of response validations that passed.	100.0 %
Virtual users	Simulated users sending requests at the same time.	1 at most
Data sent	Data uploaded to the API.	12.8 KB
Data received	Data downloaded from the API.	51.0 KB

Query language	Requests	Error rate	p50	p95
en	20	0.00 %	1,275 ms	3,994 ms
es	3	0.00 %	1,314 ms	1,410 ms

Analysis charts

Latency over time

Duration of every request, in the order they were sent, with the rolling median.

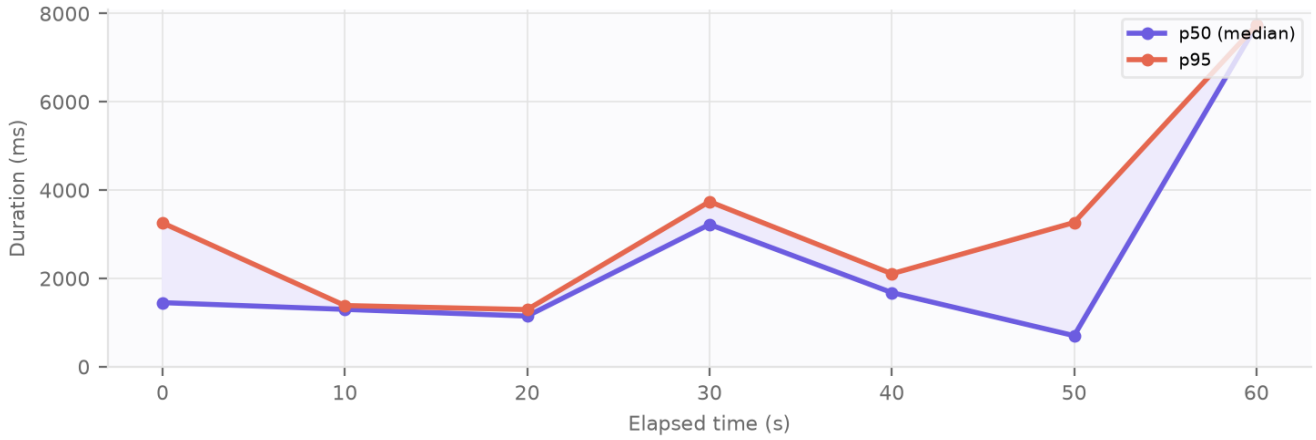


Median 1,314 ms · p95 3,789 ms · max 7,735 ms.

How to read it: A rising median while the test progresses points to saturation. Isolated high dots are slow requests (for example cold starts).

Latency percentiles over time

Median (p50) and p95 latency in each time window of the test.

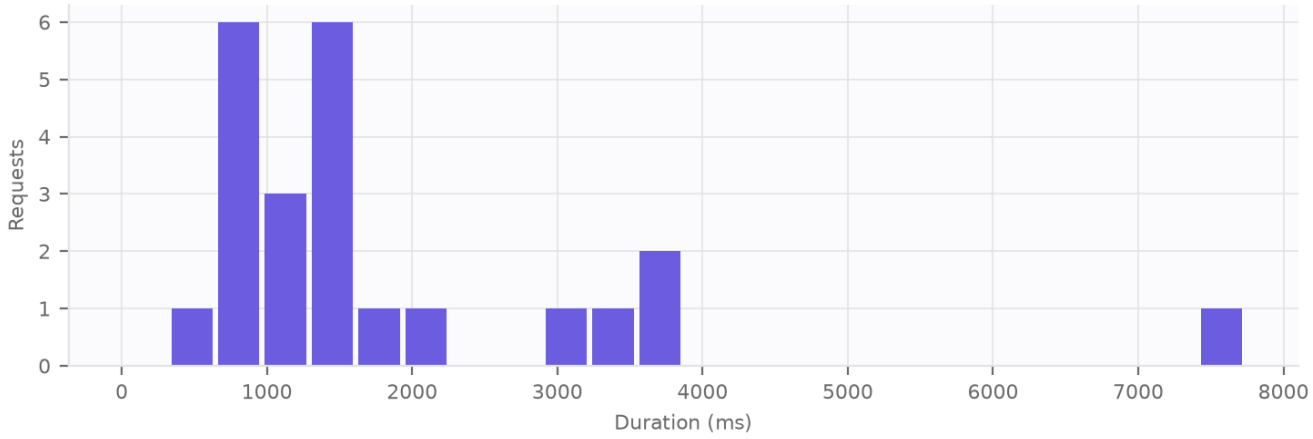


Windows of 10 s.

How to read it: The median shows the typical request; p95 shows the slow ones. A growing gap between both lines means latency is becoming less predictable.

Latency distribution

Number of requests in each duration range.

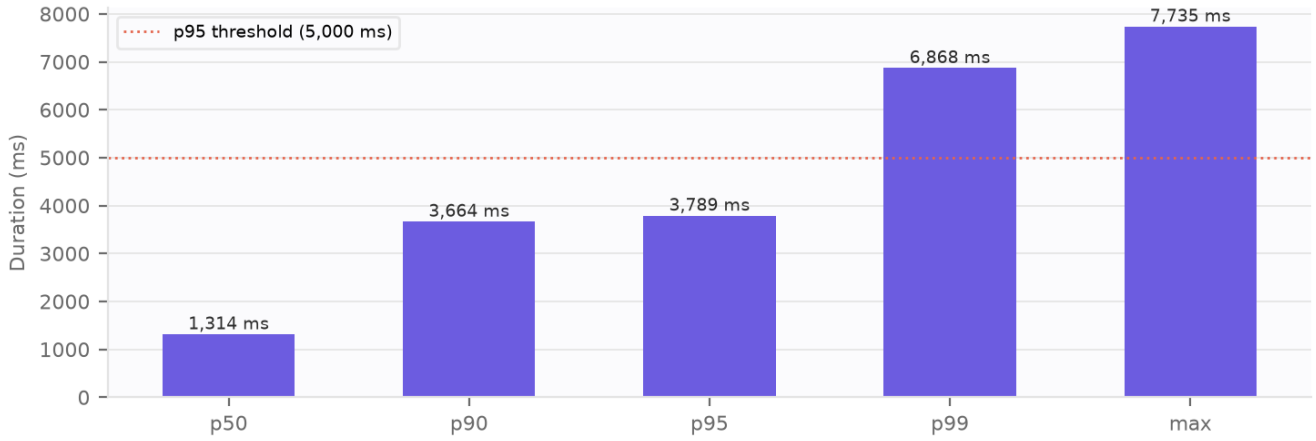


23 requests between 644 ms and 7,735 ms.

How to read it: A narrow distribution means stable latency; a long tail to the right is a group of slow requests that raises p95 and p99.

Latency percentiles

Latency below which 50%, 90%, 95% and 99% of the requests finished, and the maximum.

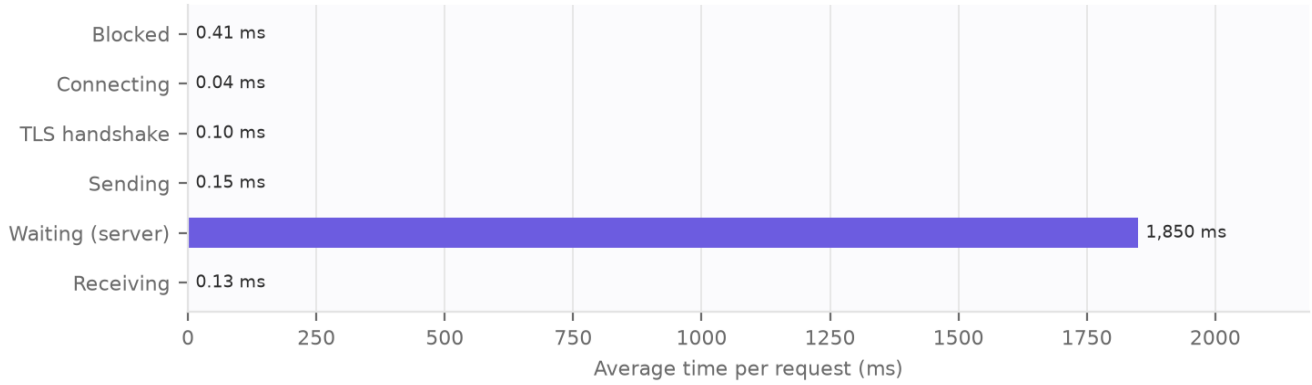


p50 1,314 ms · p90 3,664 ms · p95 3,789 ms · p99 6,868 ms · threshold p95 < 5,000 ms.

How to read it: The gap between p50 and p99 measures how unpredictable the latency is. Compare p95 with the threshold of the scenario.

Where the time goes

Average time of each phase of a request: connection, sending, waiting for the server and receiving.

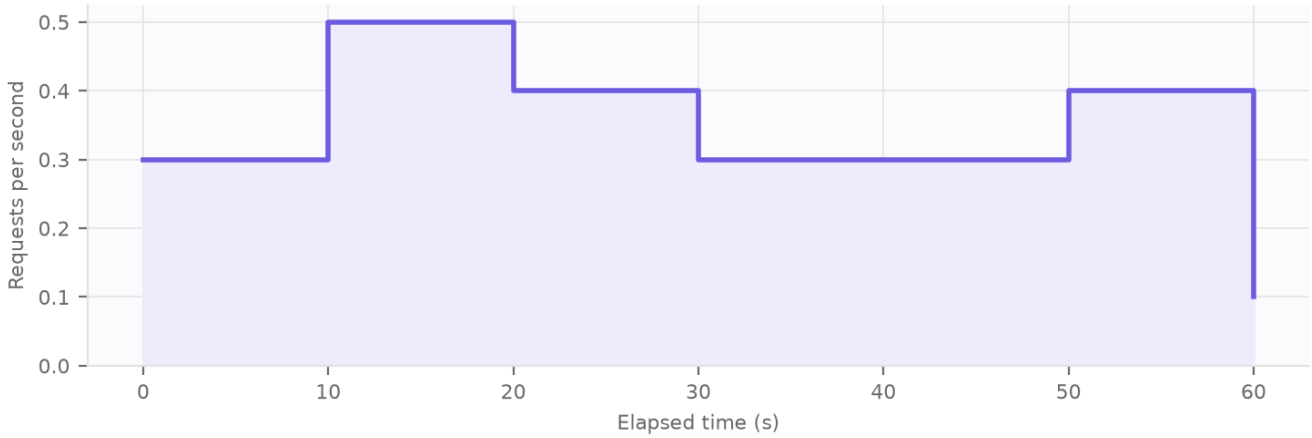


Waiting time (server processing) is >99.9 % of the request time.

How to read it: If waiting time dominates, the time is spent processing in the backend (Lambda and Bedrock), not in the network.

Throughput over time

Requests completed per second in each time window.

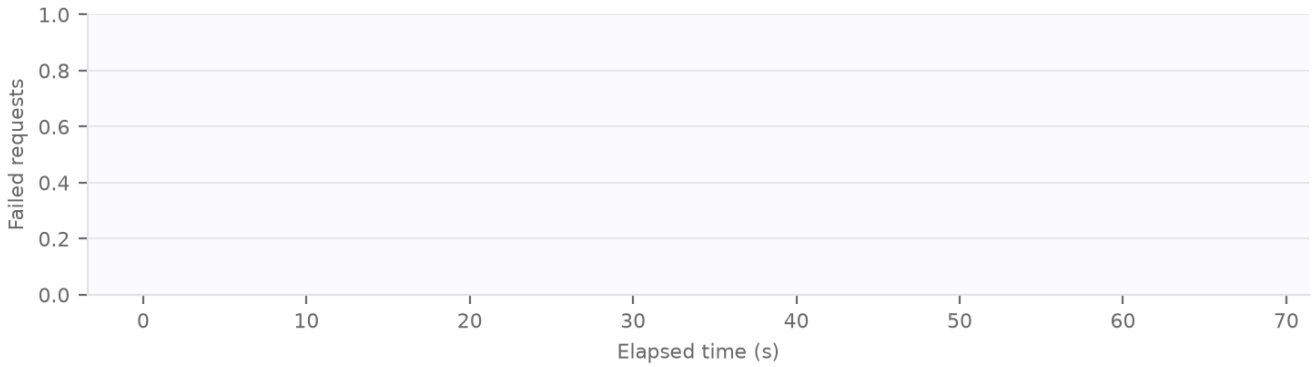


Average 0.35 requests per second.

How to read it: It should follow the number of virtual users; if it stops growing while users increase, the system is saturated.

Failed requests over time

Number of failed requests in each time window.

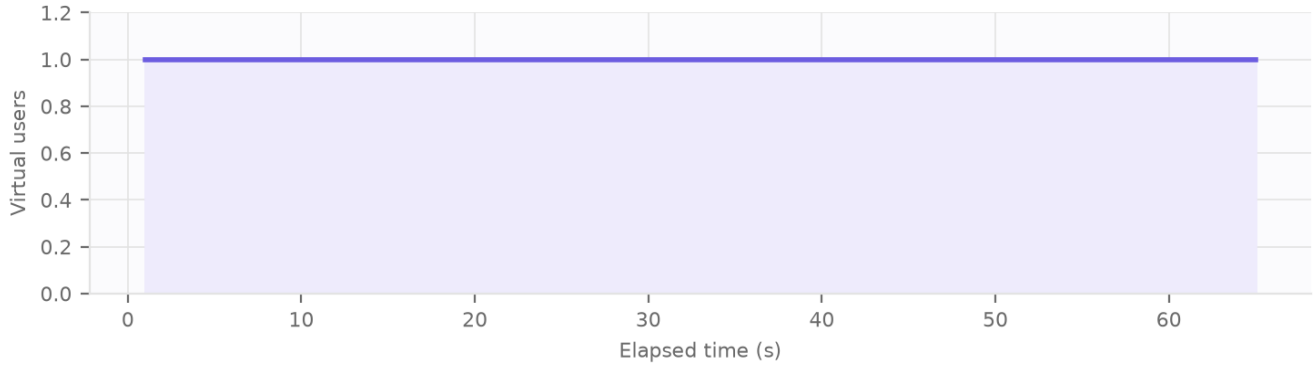


0 failed requests in total.

How to read it: Errors that appear only at the highest load usually mean throttling or timeouts.

Virtual users over time

Number of simulated users sending requests during the test.

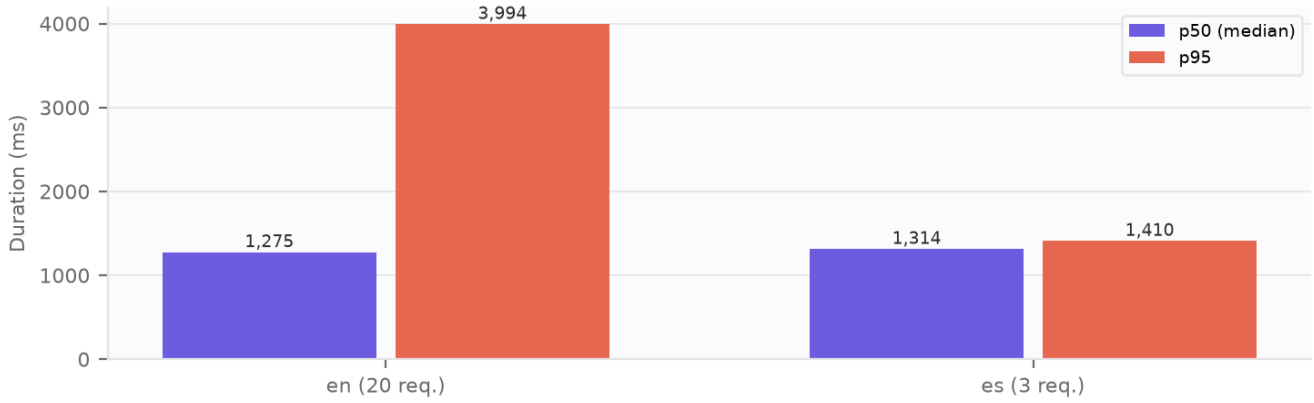


Up to 1 virtual users.

How to read it: Read the other charts against this one: changes in latency or errors usually coincide with changes in load.

Latency by query language

Median and p95 latency of the queries written in each language.



How to read it: Large differences suggest that some queries need more work in the backend (for example, the extraction model).

AI-generated summary

The smoke test passed with no errors and all thresholds met, but the 95th percentile latency is close to the threshold.

1. OVERALL ASSESSMENT

The smoke test executed 23 requests over 1 minute with 1 virtual user, resulting in an average latency of 1850.11 ms and a 95th percentile latency of 3789.09 ms. The test passed all thresholds, with no failed requests and a 95th percentile latency below 5000 ms. However, the 95th percentile latency is close to the threshold, indicating potential performance issues under higher load.

2. KEY FINDINGS

- **High 95th Percentile Latency (warning):** The 95th percentile latency of 3789.09 ms is close to the 5000 ms threshold, suggesting potential performance bottlenecks.
- **Small Sample Size (info):** With only 23 requests, the 95th and 99th percentile latencies may not be reliable.
- **No Failed Requests (info):** The error rate was 0%, indicating no failed requests during the test.

3. RECOMMENDATIONS

- Monitor the 95th percentile latency closely as load increases.
- Consider running additional tests with higher virtual user counts to identify potential bottlenecks.

4. CAVEATS

- The small sample size may affect the reliability of the 95th and 99th percentile latencies.

Validation of the AI answer

- Response structure: valid.
- Figures quoted by the model and checked against the computed metrics: 7.
- All checked figures match the computed metrics.
- Confidence reported by the model: medium.

Please read before use

- This section is generated by Amazon Bedrock (us.amazon.nova-pro-v1:0) from the metrics computed in this report. The model interprets numbers; it does not measure anything.
- The tables and charts are the source of truth: always check the AI text against them.
- Causes suggested by the model are hypotheses consistent with the data, not confirmed diagnoses.

Annex · Metric glossary

HTTP requests (http_reqs, count)

Total number of HTTP requests sent during the test. It reflects the load generated against the API.

Calculation: Each request sent adds 1 to the count. The rate is the count divided by the test duration.

Example: 100 requests in 60 seconds give a count of 100 and a rate of 1.67 requests per second.

Why it matters: It shows the scale of the test: results are only comparable between runs with a similar load.

Failed requests (http_req_failed, %)

Share of requests that returned an error status (4xx or 5xx) or did not get a response.

Calculation: Failed requests divided by total requests.

Example: 2 failed requests out of 200 give an error rate of 1%.

Why it matters: It is the first indicator of a problem: throttling, timeouts or errors in the API.

Request duration (latency) (http_req_duration, ms)

Total time of an HTTP request: sending the request, waiting for the server and receiving the response.

Calculation: Sending time + waiting time + receiving time.

Example: Send 1 ms + wait 1,200 ms + receive 1 ms = 1,202 ms.

Why it matters: It is the latency the client experiences. Percentiles (p95, p99) show the slowest requests, which an average hides.

Waiting time (TTFB) (http_req_waiting, ms)

Time from sending the last byte of the request until receiving the first byte of the response (Time To First Byte).

Calculation: First response byte received - last request byte sent.

Example: Request sent at 100 ms, first response byte at 1,300 ms: waiting time is 1,200 ms.

Why it matters: If it is close to the total duration, the bottleneck is the backend, not the network.

Blocked time (http_req_blocked, ms)

Time spent waiting for a free TCP connection before the request could be sent.

Calculation: Connection slot available - request ready to send.

Example: Request ready at 0 ms, connection available at 5 ms: blocked time is 5 ms.

Why it matters: High values point to client-side or network congestion.

Connecting time (http_req_connecting, ms)

Time to establish the TCP connection with the server.

Calculation: Connection established - connection started.

Example: Connection started at 0 ms and established at 20 ms: connecting time is 20 ms.

Why it matters: It is usually only paid on the first request, because connections are reused.

TLS handshake time (http_req_tls_handshaking, ms)

Time spent negotiating the secure (HTTPS) connection.

Calculation: Handshake completed - handshake started.

Example: Handshake started at 20 ms and completed at 60 ms: TLS time is 40 ms.

Why it matters: Like connecting time, it is mostly paid once per connection.

Sending time (http_req_sending, ms)

Time to send the request body to the server.

Calculation: Last byte sent - first byte sent.

Example: A small JSON body is usually sent in less than 1 ms.

Why it matters: Only relevant for large request bodies or slow networks.

Receiving time (http_req_receiving, ms)

Time to download the response, from its first byte to its last byte.

Calculation: Last response byte - first response byte.

Example: A response of a few kilobytes is usually received in less than 1 ms.

Why it matters: High values point to large responses or a slow network.

Iteration duration (iteration_duration, ms)

Time to complete one full iteration of the test script, including the pause between requests (think time).

Calculation: Request duration + checks + think time.

Example: A 1,200 ms request plus 1 s of think time gives an iteration of about 2,200 ms.

Why it matters: It explains the throughput: each virtual user only starts a new request when its iteration ends.

Virtual users (vus, users)

Number of simulated users sending requests at the same time.

Calculation: Defined by the scenario (constant or ramping).

Example: The stress scenario ramps from 5 to 40 users.

Why it matters: It is the load applied: latency and errors must always be read together with the number of users.

Checks (checks, %)

Share of validations on each response that passed (for example: status 200, request not forbidden).

Calculation: Passed checks divided by total checks.

Example: 26 requests with 3 checks each give 78 checks; if all pass, the rate is 100%.

Why it matters: A request can return 200 and still be wrong: checks validate the response, not only the status.

Data received (data_received, bytes)

Total amount of data received in the responses.

Calculation: Sum of the size of all responses.

Example: 26 responses of about 2.3 KB each give about 60 KB.

Why it matters: Large responses increase receiving time and network cost.

Data sent (data_sent, bytes)

Total amount of data sent in the requests, including headers.

Calculation: Sum of the size of all requests.

Example: 26 requests of about 580 bytes each give about 15 KB.

Why it matters: Signed requests add headers, so each request is larger than its body.